КАК УСТРОЕНО ПРИЛОЖЕНИЕ «МАРШРУТ»

Памятка для подготовки к защите: зачем нужно приложение, что делает каждый экран, где что лежит в коде и как коротко ответить на вопрос преподавателя.

1. Идея в двух словах

«Маршрут» сам составляет личный план обучения. Пользователь выбирает, что хочет изучить (одно из четырёх направлений), проходит короткий тест, указывает, сколько часов в неделю готов заниматься и к какому сроку нужно успеть. Приложение убирает темы, которые человек уже знает, ставит остальные в правильном порядке (сначала основы, потом то, что на них опирается), раскладывает их по дням и добавляет повторения. Если человек пропустил занятия, план перестраивается одной кнопкой.

Приложение состоит из двух программ. **Сервер API** запускается на ноутбуке и хранит все аккаунты, планы, занятия и заметки в базе SQLite. **Приложение** на телефоне ничего не хранит у себя: оно получает, добавляет, изменяет и удаляет данные только запросами к серверу по Wi-Fi (это требование задания на курсовой проект). Интернет и облачные сервисы не нужны. Приложение написано на Flutter, поэтому один и тот же код собирается для Android, iPhone и Windows; сервер написан на Dart и собирается в программу для Windows.

В каталоге 4 направления: «Flutter-разработчик» (20 тем), «Python с нуля» (18), «Английский A1–B1» (19) и «Алгоритмы и структуры данных» (18). Всего 75 тем, 113 связей «эту тему изучать после той» и 32 вопроса для теста. Каталог лежит в файлах `assets/catalog/*.json`: сервер читает их при запуске и отдаёт приложению по запросу `GET /api/directions`, поэтому новое направление добавляется без изменения кода.

2. Экраны по порядку

Таблица 1 — Экраны приложения в том порядке, в каком их видит пользователь

Экран	Что видит и делает пользователь	Файл
Заставка (Splash)	Логотип «M» и название. Приложение загружает с сервера каталог тем и проверяет, отмечал ли пользователь «Запомнить меня»: если да и сервер подтвердил вход, сразу открывается главная, иначе — вход. Если сервер выключен, заставка не зависает и открывает вход	`screens/splash_screen.dart`
Регистрация	Имя, email, пароль и повтор пароля. Ошибки показываются под полями: пустое поле, неверный email, слабый пароль, пароли не совпадают, email уже занят	`screens/auth/register_screen.dart`
Вход	Email и пароль, флажок «Запомнить меня», переход к регистрации. При неверных данных — сообщение «Неверный email или пароль». Внизу кнопка «Сервер: …»: в окне вводится адрес сервера, кнопка «Проверить и сохранить» проверяет связь	`screens/auth/login_screen.dart`, `widgets/server_address.dart`
Онбординг, шаг 1: направление	Выбор одного из четырёх направлений	`screens/onboarding/direction_screen.dart`
Шаг 2: тест	Вопросы по темам направления, вариант «Не знаю». Новичок может пропустить тест кнопкой «Я новичок»	`screens/onboarding/diagnostic_screen.dart`
Шаг 3: расписание	Часы в неделю, учебные дни, дата начала, дедлайн (можно без срока). Кнопка «Сформировать план»	`screens/onboarding/schedule_screen.dart`
Шаг 4: предпросмотр	Сколько тем и часов, какие темы убраны по тесту, дата окончания. Если в дедлайн не успеть — подсказка «нужно N ч в неделю» и кнопка, которая сразу ставит это значение. Кнопка «Начать обучение» сохраняет план	`screens/onboarding/preview_screen.dart`
Главная	Приветствие, кольцо прогресса, серия дней подряд, сколько дней до дедлайна, занятия на сегодня с отметкой. Если есть пропуски — баннер «Пропущено N занятий» с кнопкой «Перестроить». Запуск Pomodoro	`screens/home/home_screen.dart`
План	Все занятия по дням. Фильтры «Сегодня», «Предстоящие», «Выполненные», «Пропущенные», «Повторения», поиск по темам. Меню: настройки плана и «Мои планы»	`screens/plan/plan_screen.dart`
Занятие	Описание темы и материалы, отметка «Выполнено» и её отмена, для повторения — подсказка «Как повторять», запуск таймера	`screens/plan/session_screen.dart`

Экран	Что видит и делает пользователь	Файл
Настройки плана	Те же поля, что в шаге 3. «Сохранить и перестроить» пересчитывает план от сегодняшнего дня	`screens/plan/plan_settings_screen.dart`
Мои планы	Несколько планов: сделать активным, переименовать, перестроить, удалить, создать новый	`screens/plan/plans_screen.dart`
Темы	Каталог направления: поиск по названию и описанию, фильтр по категориям, вкладки «Избранное» (звёздочка) и «Мои» (свои темы)	`screens/topics/topics_screen.dart`
Тема и своя тема	Описание, материалы, от каких тем зависит, заметки к теме. Свою тему можно создать, изменить, удалить и добавить в активный план — план перестроится	`screens/topics/topic_screen.dart`, `custom_topic_screen.dart`
Статистика	Часы учёбы, число занятий, число «помидоров». График часов по неделям, круговая диаграмма по категориям, история выполненных занятий	`screens/stats/stats_screen.dart`
Профиль	Аватар, имя, email, дата регистрации, итоги. Редактирование профиля, смена пароля, тема оформления (светлая, авто, тёмная), ежедневное напоминание, строка «Сервер API» с адресом сервера, «Мои заметки», «Мои планы», выход, удаление аккаунта	`screens/profile/`
Заметки	Список заметок с поиском, создание, изменение и удаление, привязка к теме	`screens/notes/`
Pomodoro	Таймер 25/5 или 50/10 минут: старт, пауза, сброс. Когда фокус закончился — уведомление, запись в статистику и перерыв	`screens/pomodoro/pomodoro_screen.dart`

Пути в таблице даны от папки `lib/presentation/`. Нижняя панель с пятью вкладками (Главная, План, Темы, Статистика, Профиль) описана в `screens/shell/main_shell.dart`; в широком окне Windows она превращается в боковую панель.

3. Как всё устроено внутри

3.1 Приложение и сервер, три слоя

Система — это две программы: приложение (Flutter) и сервер API (Dart). Они общаются по протоколу HTTP, данные передаются в формате JSON. Схема — рисунок 2.1 курсовой.

Код приложения разделён на три слоя по принципам чистой архитектуры. Зависимости направлены внутрь: интерфейс и данные знают о предметной области, а предметная область ни о ком не знает.

- **`lib/domain` — предметная область.** Чистый Dart без Flutter и базы данных: сущности (тема, план, занятие, настройки расписания), алгоритм построения плана (`planner/`), интерфейсы репозиториев (`repositories/repositories.dart`) и сценарий работы с планом `usecases/plan_service.dart`. Поэтому алгоритм проверяется быстрыми модульными тестами.
- **`lib/data` — данные.** Приложение использует папку `api/`: клиент API (`api_client.dart`) и репозитории, каждый метод которых отправляет запрос серверу. Папки `db/` (база SQLite через пакет drift) и `repositories/` использует сервер. Здесь же уведомления (`services/reminder_service_impl.dart`) и хранение адреса сервера и токена входа (`repositories/session_storage_impl.dart`).
- **`lib/presentation` — интерфейс.** Экраны (`screens/`), общие виджеты (`widgets/`) и провайдеры Riverpod (`providers/`), которые связывают экраны с данными.
- **`lib/server` и `bin/` — сервер API.** `server/api_server.dart` — все адреса API, проверка токена и данных, коды ответов и журнал; `bin/marshrut_server.dart` — запуск сервера.
- **`lib/core` — общее:** маршруты и правило доступа (`router/app_router.dart`), тема оформления (`theme/app_theme.dart`), хэширование паролей и проверка полей (`utils/`).

Состоянием управляет **Riverpod**: экран не создаёт объекты сам, а берёт их у провайдеров (`providers/providers.dart`). В тестах провайдеры подменяются: сервер API с базой в памяти, фиксированная дата, быстрое хэширование.

3.2 Как строится план

Алгоритм лежит в `lib/domain/planner/` и работает в пять шагов (`plan_generator.dart`):

1. **Граф тем.** Темы направления — это вершины, а связь «В изучать после А» — ребро. Получается ориентированный граф без циклов.
2. **Исключение известных тем** (`knowledge_estimator.dart`). Верный ответ в тесте означает, что известны сама тема и все темы, на которых она стоит. Неверный ответ или «Не знаю» означает, что тема не известна, даже если её «покрыл» верный ответ по более сложной теме.
3. **Порядок тем** — топологическая сортировка, алгоритм Кана (`topological_sort.dart`). Берутся темы, у которых не осталось неизученных предшественников; среди них первой идёт более простая. Если в данных цикл, алгоритм это обнаруживает и сообщает об ошибке.
4. **Раскладка по дням** (`scheduler.dart`). Дневной лимит — часы в неделю, делённые на число учебных дней (округляется до 15 минут, не меньше 30). День за днём: сначала повторения, срок которых наступил (по 15 минут), потом текущая тема; большая тема делится на части не короче 30 минут. После каждой изученной темы ставятся повторения через 1, 3 и 7 дней — по кривой забывания Эббингауза.
5. **Проверка дедлайна.** Если последнее занятие выходит за срок, приложение подбирает с шагом 0,5 часа минимальное число часов в неделю, при котором план успевает (не больше 84 часов, это 12 часов в день).

Пример из курсовой (таблица 2.5): три темы по 60 минут, «Основы Dart» → «Виджеты» → «Состояние», 7 часов в неделю, все дни, дедлайн через три дня. Дневной лимит — 60 минут. 1-й день: «Основы Dart» 60 минут. 2-й день: повторение «Основ Dart» 15 минут и первая часть «Виджетов» 45 минут. 3-й день: вторая часть «Виджетов» 15 минут и первая

часть «Состояния» 45 минут. Последняя часть «Состояния» попадает на 4-й день, то есть позже дедлайна, поэтому приложение предлагает 8 часов в неделю: лимит становится 75 минут, и план укладывается в срок.

3.3 Перепланирование

`replanner.dart` вызывается кнопкой «Перестроить» или после изменения расписания. Выполненные занятия остаются на своих местах. Для каждой темы считается, сколько минут уже изучено, остаток заново раскладывается с сегодняшнего дня теми же правилами, а пропущенные повторения переносятся на ближайшие учебные дни. Номера частей тем продолжаются: если изучены части 1 и 2, следующей будет часть 3.

3.4 База данных

База SQLite находится на сервере, в файле `marshrut_server.sqlite` рядом с программой-сервером. Описание таблиц — `lib/data/db/app_database.dart`. Таблиц девять: пользователи (`users`), планы (`plans`), темы плана в порядке изучения (`plan_topics`), занятия (`sessions`), свои темы (`custom_topics`), заметки (`notes`), избранное (`favorites`), записи Pomodoro (`pomodoro_logs`) и токены входа (`api_tokens`). Все связи объявлены внешними ключами с каскадным удалением: удалили план — удалились его занятия, удалили аккаунт — удалились все данные пользователя. Сам каталог тем в базе не хранится, он читается из JSON.

На телефоне хранятся только настройки, в `shared_preferences` (`session_storage_impl.dart`): адрес сервера, кто вошёл (id и токен входа) и выбранная тема оформления.

3.5 Сервер API и обработка ошибок

Сервер (`lib/server/api_server.dart`, библиотеки shelf и shelf_router) принимает запросы на порту 8080. У каждого адреса свой метод: GET —

получить, POST — добавить, PUT и PATCH — изменить, DELETE — удалить. Всего 39 адресов (таблица в приложении К курсовой и в `docs/Сервер_API.md`). Полный набор «получить, добавить, изменить, удалить» есть у заметок, своих тем и планов.

Каждый запрос проходит одну цепочку: строка в журнале окна сервера → перехват ошибок → выбор обработчика по методу и адресу → проверка токена → проверка данных теми же правилами, что и в формах приложения → проверка, что запись принадлежит этому пользователю → чтение или изменение базы. Ответ — JSON и код: 200, 201 (создано), 204 (выполнено), 400 (неверные данные), 401 (нужен вход), 404 (не найдено или чужая запись), 500 (ошибка сервера).

В приложении все запросы проходят через `lib/data/api/api_client.dart`. Клиент добавляет токен, ждёт ответ не дольше 10 секунд и превращает любую неудачу в понятное сообщение: «Нет связи с сервером…», «Сервер … не отвечает», текст ошибки от сервера. При ответе 401 приложение само возвращает на экран входа. Если действие не удалось (отметить занятие, сохранить заметку), внизу экрана появляется красное сообщение (функция `runAction` в `widgets/common.dart`), а данные не меняются. После каждого изменения клиент сообщает экранам, что данные обновились, и они загружают их заново.

3.6 Регистрация, вход и пароли

Приложение отправляет форму на сервер (`lib/data/api/api_auth_repository.dart`), а сервер проверяет и сохраняет данные (`lib/data/repositories/auth_repository_impl.dart`, `lib/core/utils/password_hasher.dart`). Пароль не хранится: в базу записываются случайная соль (16 байт) и хэш PBKDF2-HMAC-SHA256 со 100 000 повторений. Расчёт идёт в отдельном изоляте, чтобы интерфейс не замирал, а хэши сравниваются за постоянное время. В первом семестре был

SHA-256 с солью; такие хэши пересчитываются в новый формат при следующем входе.

После входа сервер выдаёт токен — 32 случайных байта. Приложение отправляет его в заголовке `Authorization: Bearer …` с каждым запросом. На сервере хранится только хэш токена; токен действует 30 дней, а при выходе удаляется. «Запомнить меня» сохраняет токен на телефоне, и при следующем запуске заставка, спросив сервер, сразу открывает главную. Состояние входа держит `providers/auth_controller.dart`.

3.7 Навигация и защита экранов

Маршруты описаны в `lib/core/router/app_router.dart` (пакет go_router). Перед каждым переходом срабатывает одна функция `authRedirect`: пока сессия проверяется, виден только Splash; без входа доступны только вход и регистрация; после входа экраны входа ведут на главную. Поэтому после выхода или удаления аккаунта закрытые экраны сразу становятся недоступны. Каждая вкладка нижней панели помнит свою прокрутку и фильтры (`StatefulShellRoute`).

3.8 Уведомления

`lib/data/services/reminder_service_impl.dart`, пакет flutter_local_notifications. На Android и iPhone работают ежедневное напоминание в выбранное время (включается в профиле) и мгновенное уведомление об окончании «помидора». В Windows плагин не умеет повторять уведомление каждый день без установщика MSIX, поэтому там переключатель напоминания неактивен с подписью «Доступно в версии для Android и iOS», а уведомление Pomodoro работает.

3.9 Pomodoro

`lib/presentation/screens/pomodoro/pomodoro_screen.dart`. Два режима: 25 минут фокуса и 5 перерыва или 50 и 10. Оставшееся время

считается от момента окончания интервала, а не уменьшением счётчика, поэтому таймер не отстаёт. Законченный интервал отправляется на сервер (`POST /api/pomodoros`), записывается в таблицу `pomodoro_logs` и попадает в статистику.

3.10 Тесты и сборка

137 модульных и виджет-тестов в папке `test/`: алгоритм, база, сервер API (`test/server/`: коды ответов, токены, чужие данные, сбой базы), клиент API (нет связи, таймаут, ответ не JSON), экраны, правило доступа, адаптивность. В виджет-тестах приложение работает с настоящим сервером API, у которого база в памяти. Интеграционный тест в `integration_test/` запускает настоящий HTTP-сервер и настоящее приложение: регистрация → вход → создание плана → профиль → выход. При каждой отправке кода GitHub Actions (`.github/workflows/build.yml`) запускает анализ и все тесты, затем собирает APK для Android, IPA для iPhone, программу для Windows и программу-сервер. Матрица из 280 снимков всех экранов (`tool/screenshots/qa_matrix_test.dart`) помогла найти ошибки вёрстки; всего найдено и исправлено 27 ошибок (`docs/coursework/testing.md`).

4. Если спросят

Таблица 2 — Короткие ответы на вероятные вопросы

Вопрос	**Ответ**
Зачем это приложение?	Чтобы человек, который учится сам, получил готовый план под своё время и срок, а не составлял его вручную
Чем отличается от roadmap.sh, Duolingo, Coursera?	Они дают схему тем, мотивацию или курсы, но ни одно не учитывает знания пользователя, его часы и дедлайн одновременно и не перестраивает план при отставании
Почему Flutter?	Один код для Android, iPhone и Windows, собственная отрисовка (везде выглядит одинаково), горячая перезагрузка, Material 3 из коробки
Что такое чистая архитектура в проекте?	Три слоя: domain (алгоритм и сущности, чистый Dart), data (клиент и репозитории API), presentation (экраны). Алгоритм не зависит от Flutter и от того, где лежат данные, поэтому при переходе на сервер его не пришлось менять
Почему Riverpod?	Проверка зависимостей при компиляции, нет зависимости от дерева виджетов, легко подменять данные в тестах

Вопрос	**Ответ**
Где хранятся данные?	В базе SQLite на сервере, который запущен на ноутбуке. Приложение работает с данными только через API сервера
Почему свой сервер, а не Firebase или Supabase?	Им нужен интернет, учётная запись и ключи, а данные лежат у чужого сервиса. Свой сервер работает в локальной сети Wi-Fi, написан на том же Dart и использует те же правила проверки, что и приложение
Что такое API и REST?	API — набор адресов, по которым приложение просит сервер что-то сделать. REST — правила: каждая запись имеет свой адрес (`/api/notes/3`), а действие задаётся методом HTTP: GET — получить, POST — добавить, PUT/PATCH — изменить, DELETE — удалить
Как приложение узнаёт об ошибке?	По коду ответа: 2xx — успех, 400 — неверные данные, 401 — нужен вход, 404 — не найдено, 500 — ошибка сервера. Текст ошибки сервер передаёт в JSON, приложение показывает его пользователю
Что будет, если выключить сервер?	Приложение покажет «Нет связи с сервером… Проверьте, что сервер запущен и телефон в той же сети Wi-Fi», данные не изменятся. Когда сервер снова включат, списки загрузятся сами
Как сервер понимает, кто прислал запрос?	По токену входа в заголовке `Authorization: Bearer …`. Без токена сервер отвечает 401, а чужие записи для него «не найдены» (404)
Почему SQLite (drift) на сервере?	Связи между таблицами и каскадное удаление есть в SQLite из коробки, база — один файл, который легко перенести; drift генерирует типобезопасный код
Как строится порядок тем?	Топологической сортировкой графа зависимостей (алгоритм Кана), сложность $O((V + E) \cdot \log V)$
Что будет, если в каталоге ошибка — цикл зависимостей?	Алгоритм Кана это обнаружит: останутся темы, которые нельзя поставить, и приложение покажет понятное сообщение
Как работает тест знаний?	Верный ответ отмечает тему и все её основы как известные, неверный ответ или «Не знаю» оставляет тему в плане
Откуда интервалы повторений 1, 3, 7 дней?	Из кривой забывания Эббингауза и эффекта распределённого повторения: повторять лучше с растущими промежутками
Что если не успеваю к дедлайну?	Приложение подбирает минимальное число часов в неделю с шагом 0,5 и предлагает установить его одной кнопкой
Что если пропустил занятия?	Кнопка «Перестроить»: выполненное остаётся, остаток раскладывается с сегодняшнего дня
Как хранятся пароли?	Только хэш PBKDF2-HMAC-SHA256 со случайной солью и 100 000 повторений; сам пароль нигде не сохраняется
Как закрыты экраны без входа?	Одной функцией redirect в роутере: без входа открываются только вход и регистрация
Где реализованы CRUD-операции?	Заметки, свои темы и планы: создание, просмотр, изменение, удаление через API (POST, GET, PUT/PATCH, DELETE), на сервере — в базе SQLite; ошибки показываются понятным сообщением

Вопрос	Ответ
Как проверяли приложение?	137 автоматических тестов (в том числе сервера и клиента API), интеграционный тест с настоящим сервером, 45 тест-кейсов, ручные проверки и 280 снимков экранов; найдено и исправлено 27 ошибок
Как установить приложение?	APK для Android, IPA для iPhone, архив для Windows и программу-сервер собирает GitHub Actions. APK ставится прямо на телефон, IPA — с компьютера программой Sideloadly (`docs/Установка_на_iPhone.md`), сервер запускается файлом «Запустить сервер» (`docs/Сервер_API.md`)